

Report secondo progetto di Metodi del calcolo scientifico

Daniele Besozzi 894998
Andrea Consonni 900116
Matteo Cervini 902225

2 giugno 2026

Indice

1	Prima parte: implementazione e benchmark DCT2	2
1.1	Metodologia di misura delle performance	2
1.2	Risultati	3
2	Seconda parte: compressione immagini	4
2.1	Test su immagini	4
2.1.1	Tony Pitony	4
2.1.2	Lena	7
3	Struttura del progetto	10
3.1	Implementazione della DCT2	11
3.2	Part1 e Part2	12
3.2.1	GUI	13

Capitolo 1

Prima parte: implementazione e benchmark DCT2

Per l'implementazione della DCT2 abbiamo scelto il linguaggio di programmazione Java, mettendo a confronto la nostra implementazione con l'implementazione fornita dalla libreria `JTransform` [1]. Per effettuare le operazioni di algebra lineare richieste in modo semplice ed ottimale, abbiamo scelto di utilizzare la libreria `Efficient Java Matrix Library`(ejml)[2].

1.1 Metodologia di misura delle performance

Per misurare le performance della nostra implementazione rispetto a quella della libreria, abbiamo generato in maniera causale matrici di dimensioni pari a 2^i , $i = 3, \dots, 13$. Le entrate di questa matrice sono generate in maniera casuale in un intervallo di valori a virgola mobile compreso tra 0 e 100.

```
1  public static double[][] randomMatrix(int n) {
2
3      double[][] m = new double[n][n];
4
5      for (int i = 0; i < n; i++) {
6          for (int j = 0; j < n; j++) {
7              m[i][j] = Math.random()*100;
8          }
9      }
10
11     return m;
12 }
```

Listing 1.1: Generazione di matrici casuali

Per eseguire le misurazioni, abbiamo utilizzato la libreria `Java Microbenchmark Harness`(JMH)[3], che ci ha permesso di eseguire i benchmark in modo affidabile e preciso, gestendo automaticamente le iterazioni di warmup e misurazione. Poiché Java esegue ottimizzazioni a runtime (JIT compilation), è necessario eseguire iterazioni di warmup prima delle misurazioni effettive. Durante la fase di warmup, il codice viene compilato e ottimizzato dalla JVM (Java Virtual Machine)[4]. Successivamente, eseguiamo molteplici run e calcoliamo la media dei tempi ottenuti da queste iterazioni, scartando i risultati della fase di warmup. Il codice di configurazione del benchmark è riportato nel listing 1.2.

```

1 public double run(Supplier<Supplier<?>> taskFactory) throws Exception {
2
3     Options opt = new OptionsBuilder()
4         .include(String.format(BenchmarkConstants.JMH_BENCHMARK_INCLUDE_TEMPLATE,
5             BenchmarkRunner.class.getSimpleName()))
6         .forks(0)
7         .warmupIterations(DEFAULT_WARMUP_ITERATIONS)
8         .measurementIterations(DEFAULT_MEASUREMENT_ITERATIONS)
9         .build();
10
11     Collection<RunResult> results;
12     try {
13         pendingFactory = taskFactory;
14         results = new Runner(opt).run();
15     } finally {
16         pendingFactory = null; // always clear to prevent stale lambda on reuse
17     }
18
19     if (results.isEmpty()) {
20         throw new CancellationException(BenchmarkConstants.BENCHMARK_ABORTED_NO_RESULTS);
21     }
22     return results.iterator().next().getPrimaryResult().getScore();
23 }

```

Listing 1.2: Configurazione del benchmark con JMH

1.2 Risultati

I risultati del benchmark sono riportati nella figura 1.1. Come si può evincere dal grafico, i tempi di

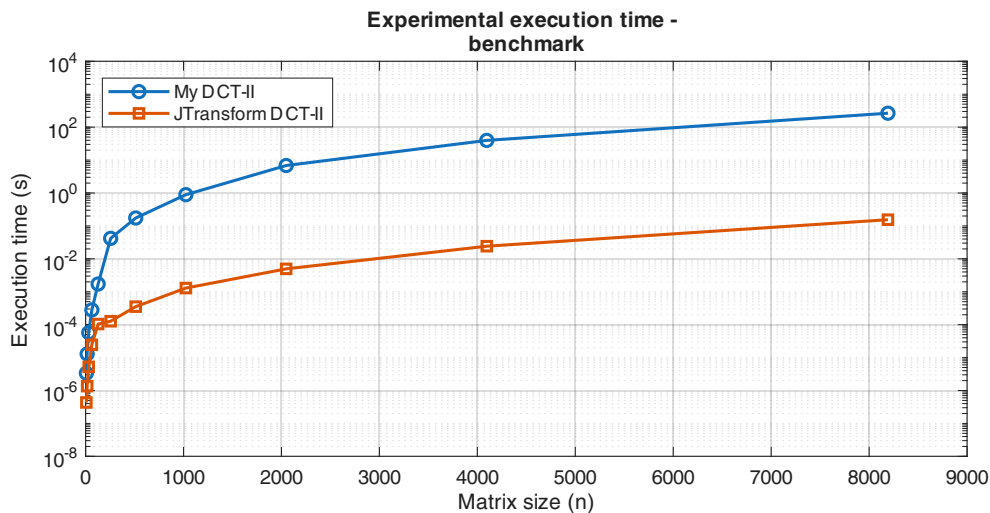


Figura 1.1: Risultati del benchmark tra la nostra implementazione della DCT2 e quella fornita da JTransform

esecuzione dell'implementazione fornita dalla libreria sono di gran lunga inferiori rispetto alla nostra implementazione. L'algoritmo implementato dalla libreria è nella versione fast (FFT) e implementa una variante dell'algoritmo dividi-et-impera per la FFT (split-radix e mixed-radix) ottimizzato per sistemi multiprocessore [5]. Ciò comporta un miglioramento significativo delle prestazioni dato sia dall'algoritmo utilizzato sia dalla capacità della libreria di sfruttare le capacità di multithreading del processore.

Capitolo 2

Seconda parte: compressione immagini

2.1 Test su immagini

Per provare la nostra implementazione della compressione tramite DCT2, abbiamo scelto due immagini: una di Tony Pitony e una di Lena (immagine classica per l'immagine processing). Per entrambe le immagini, abbiamo applicato la DCT2 con diversi valori di f (la dimensione del blocco) e d (il numero di coefficienti da mantenere). Di seguito sono riportati i risultati ottenuti per ciascuna immagine, le quali sono anche disponibili in formato .bmp al [link](#).

2.1.1 Tony Pitony

Per quanto riguarda l'immagine in figura 2.1 abbiamo provato a modificare la dimensione dei blocchi f e il coefficiente del taglio d . I valori di f che abbiamo scelto sono stati 8, 16, 32, 64 e 128, mentre i valori di d sono stati rispettivamente 4, 8, 16, 32 e 64, ovvero la metà della dimensione del blocco. Fare benchmark aumentando la dimensione dei blocchi può risultare interessante per questa immagine perché contiene sia aree uniformi sia dettagli ad alta frequenza, come i bordi degli occhiali e la texture della camicia. Blocchi più grandi migliorano generalmente la compressione nelle zone lisce, ma possono introdurre artefatti più evidenti e perdita di dettaglio nelle regioni ricche di dettagli.



Figura 2.1: Immagine di Tony Pitony non compressa



Figura 2.2: Immagine di Tony Pitony compressa con DCT2, con $f = 8$ e $d = 4$



Figura 2.3: Immagine di Tony Pitony compressa con DCT2, con $f = 16$ e $d = 8$



Figura 2.6: Immagine di Tony Pitony compressa con DCT2, con $f = 128$ e $d = 64$



Figura 2.4: Immagine di Tony Pitony compressa con DCT2, con $f = 32$ e $d = 16$



Figura 2.5: Immagine di Tony Pitony compressa con DCT2, con $f = 64$ e $d = 32$

All'aumentare della dimensione dei blocchi, possiamo vedere che la qualità dell'immagine compressa peggiora particolarmente nelle aree con molti dettagli. In particolare, si può anche notare l'introduzione di artefatti di tipo ringing nelle zone uniformi, come la finestra, che diventano più evidenti con blocchi più grandi. Un esempio di questi artefatti è mostrato in figura 2.7, dove si possono vedere delle bande scure e chiare intorno ai bordi, tipiche del fenomeno di ringing. L'effetto sulle nostre immagini è evidenziato in figura 2.8, dove si possono notare le bande scure attorno alla gamba e ai bordi della finestra.



Figura 2.7: Artefatti di tipo ringing



Figura 2.8: Zoom sull'immagine di Tony Pitony compressa con DCT2, con $f = 128$ e $d = 64$, evidenziando gli artefatti di tipo ringing

2.1.2 Lena

Per quanto riguarda l'immagine in figura 2.9 abbiamo provato a mantenere fissa la dimensione dei blocchi f a 8 e a modificare il coefficiente del taglio d con i valori 12, 8, 4, 2 e 0. In particolare, con $d = 0$ abbiamo eliminato tutti i coefficienti, ottenendo così un'immagine completamente nera. Si può osservare che, come ci si aspetta, all'aumentare del numero di coefficienti mantenuti, la qualità dell'immagine compressa migliora, con una maggiore fedeltà all'immagine originale. In particolare, con $d = 12$ e $d = 8$ l'immagine compressa è molto simile all'originale, mentre con $d = 4$ si notano perdite di dettagli e si manifesta il tipico effetto di jpeg dove sono ben visibili i blocchi di dimensione 8×8 .



Figura 2.9: Immagine di Lena non compressa



Figura 2.10: Immagine di Lena compressa con DCT2, con $f = 8$ e $d = 12$



Figura 2.11: Immagine di Lena compressa con DCT2, con $f = 8$ e $d = 8$



Figura 2.12: Immagine di Lena compressa con DCT2, con $f = 8$ e $d = 4$



Figura 2.13: Immagine di Lena compressa con DCT2, con $f = 8$ e $d = 2$

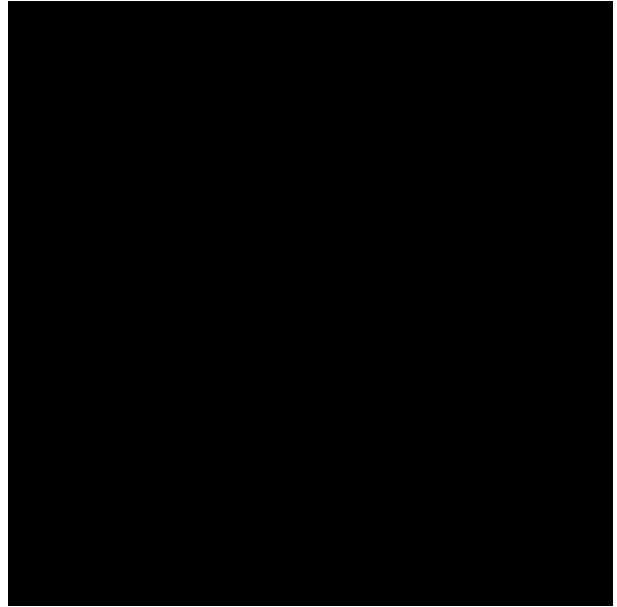


Figura 2.14: Immagine di Lena compressa con DCT2, con $f = 8$ e $d = 0$

Capitolo 3

Struttura del progetto

La struttura del progetto è organizzata in due macro package, il package **GUI** (Graphical User Interface) si occupa unicamente di gestire l'interfaccia utente, mentre il package **backend** contiene l'implementazione della nostra DCT2, la configurazione dei benchmark, la logica per la compressione delle immagini (con la DCT2 nel formato fast offerta dalla libreria esterna **JTransform**) e la generazione dei risultati in formato CSV per i benchmark e immagini in formato BMP per la compressione delle immagini.

La figura 3.1 mostra la struttura del progetto.

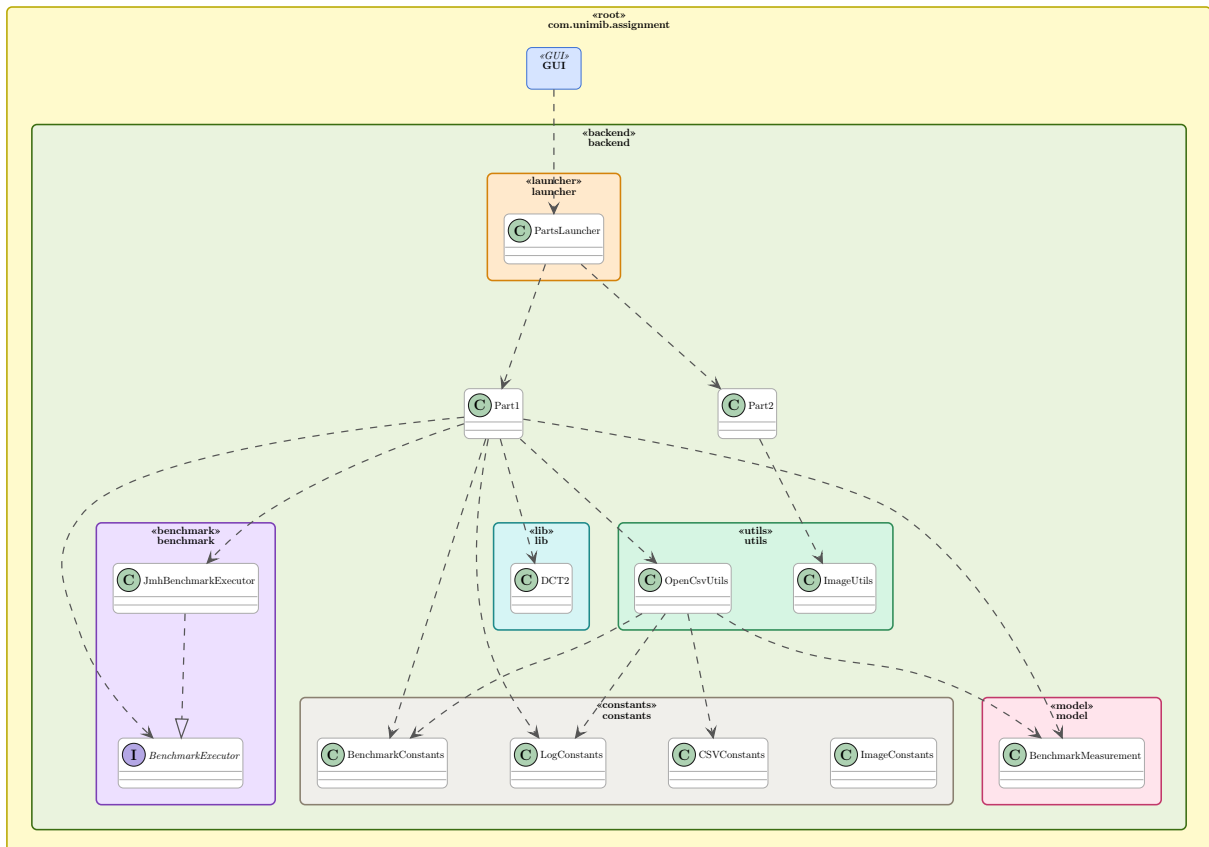


Figura 3.1: Struttura del progetto

3.1 Implementazione della DCT2

Il package `com.unimib.assignment.backend.lib` contiene l'implementazione della DCT2, questa classe implementa un metodo chiamato DCT-II che funge da algoritmo core per i due metodi offerti:

```
1      */
2      private SimpleMatrix DCTII(SimpleMatrix signal, SimpleMatrix Dn, SimpleMatrix Dm){
3          // Create a copy of the input signal to avoid modifying the caller's matrix
4          SimpleMatrix result = signal.copy();
5
6          // First pass: Apply column-wise transformation (multiply each column by Dn)
7          // This corresponds to transforming along the column dimension
8          for (int i = 0; i < result.getNumCols(); i++) {
9              // Extract the i-th column as a vector(false flag is used to extract column)
10             SimpleMatrix col = result.extractVector(false, i);
11             // Apply the transform: Dn * column
12             SimpleMatrix transformed = Dn.mult(col);
13             // Insert the transformed column back into the result matrix
14             result.insertIntoThis(0, i, transformed);
15         }
16
17         // Second pass: Apply row-wise transformation (multiply each row by Dm)
18         // This corresponds to transforming along the row dimension
19         for (int i = 0; i < result.getNumRows(); i++) {
20             // Extract the i-th row as a vector(true flag is used to extract row)
21             SimpleMatrix row = result.extractVector(true, i);
22             // Apply the transform: (Dm * (row)^T)^T = row * Dm^T
23             SimpleMatrix transformed = Dm.mult(row.transpose()).transpose();
24             // Insert the transformed row back into the result matrix
25             result.insertIntoThis(i, 0, transformed);
26         }
27
28         return result;
29     }
```

Listing 3.1: Algoritmo DCTII(DCT2) implementato nel package lib

Questa implementazione è utilizzata sia per il metodo `forward(DCT2)`:

```
1      */
2      public SimpleMatrix forward(SimpleMatrix signal) {
3          // Compute DCT basis matrices for the row and column dimensions
4          SimpleMatrix Dn = computeDMatrix(signal.getNumRows());
5          SimpleMatrix Dm = computeDMatrix(signal.getNumCols());
6          // Apply the forward DCT transformation
7          return DCTII(signal, Dn, Dm);
8      }
```

Listing 3.2: Algoritmo forward implementato nel package lib

che per il metodo `inverse(IDCT2)`:

```
1      */
2      public SimpleMatrix inverse(SimpleMatrix signal) {
3          // Compute DCT basis matrices for the row and column dimensions
4          SimpleMatrix Dn = computeDMatrix(signal.getNumRows());
5          SimpleMatrix Dm = computeDMatrix(signal.getNumCols());
6          // Apply the inverse DCT transformation
7          // (using transposed basis matrices to perform the inverse operation)
8          return DCTII(signal, Dn.transpose(), Dm.transpose());
9      }
```

Listing 3.3: Algoritmo inverse implementato nel package lib

Questo approccio ci permette di avere un codice più pulito e leggibile, evitando la duplicazione del codice per i due metodi. ¹

¹L'implementazione della DCT2 si appoggia alla libreria EJML, questo permette di avere un codice più leggibile rispetto a un'implementazione basata su matrici

3.2 Part1 e Part2

Il package `com.unimib.assignment.backend.launcher` offre la classe `PartsLauncher`, che espone alla UI due metodi: `launchAndHandlePart1`, per l'avvio dei benchmark, e `launchPart2`, per la compressione delle immagini.

launchAndHandlePart1 si occupa di invocare la classe `Part1`, responsabile della gestione del processo di esecuzione dei benchmark e della memorizzazione dei risultati.

Come prima operazione, richiama il metodo `randomMatrix` per generare le matrici casuali da utilizzare nei benchmark. Tali matrici vengono create prima dell'esecuzione dei test e condivise tra il benchmark della nostra implementazione della DCT2 e quello della libreria, così da garantire risultati confrontabili. Le dimensioni delle matrici sono memorizzate nell'array di interi `BENCHMARK_BLOCK_SIZES`.

Per l'esecuzione dei benchmark, la classe `Part1` si occupa di:

1. invocare la classe `JmhBenchmarkExecutor`, responsabile della configurazione e dell'esecuzione dei benchmark tramite JMH. Tale classe gestisce le iterazioni di warmup (`DEFAULT_WARMUP_ITERATIONS = 3`) e quelle di misurazione (`DEFAULT_MEASUREMENT_ITERATIONS = 5`), salvando i risultati nel record `BenchmarkMeasurement(size, meanTimeMine, meanTimeLib)`. I tempi sono memorizzati in secondi;
2. salvare i risultati contenuti nel record `BenchmarkMeasurement` in un file CSV utilizzando la classe `OpenCsvUtils`, che si appoggia alla libreria open-source `OpenCSV` [6];
3. gestire eventuali richieste di terminazione forzata del benchmark provenienti dalla UI, interrompendone l'esecuzione.

launchPart2 Si occupa di eseguire la compressione delle immagini utilizzando la trasformata DCT2 fornita dalla libreria `JTransform`, di inviare l'immagine compressa alla UI e di salvare le immagini compresse in formato BMP.

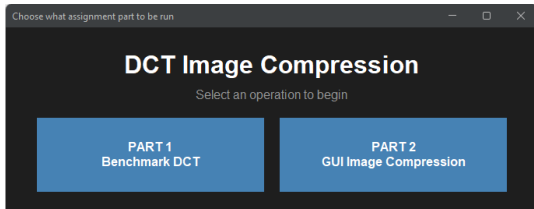
La classe `Part2` contiene i seguenti metodi:

- `BufferedImage compress(Pair<String, BufferedImage> imageInfo, int F, int d)`: esegue l'intera pipeline di compressione dell'immagine:
 1. converte l'immagine di input in una matrice di `double`;
 2. ritaglia la matrice in modo che le sue dimensioni siano multipli di `F`;
 3. per ogni blocco di dimensione $F \times F$, richiama il metodo `compressBlock` per effettuare la compressione;
 4. converte la matrice risultante in una `BufferedImage`;
 5. salva l'immagine compressa nella cartella output e successivamente la inoltra alla UI.
- `void compressBlock(double[][] signal, int i, int j, int F, int d)`: comprime un blocco dell'immagine di dimensione $F \times F$:
 1. copia il blocco in una matrice di `double`;
 2. applica la DCT2 al blocco utilizzando il metodo `forward(matrix, scale)` dell'oggetto `DoubleDCT_2D` della libreria `JTransform`, impostando `scale = true`. In questo modo viene utilizzata una normalizzazione ortonormale, coerente con quella adottata nella nostra implementazione della DCT2;
 3. elimina le componenti ad alta frequenza in base al parametro `d`;
 4. applica la trasformata inversa IDCT2 mediante il metodo `inverse(matrix, scale)`, con `scale = true`;
 5. richiama il metodo `shiftBlockBy255` per riportare i valori del blocco nell'intervallo `[0, 255]`;
 6. copia il blocco compresso nella matrice dell'immagine risultante.
- `void shiftBlockBy255(double[][] matrix)`: effettua uno shift dei valori nell'intervallo `[0, 255]`, riportandoli nel dominio delle immagini BMP.

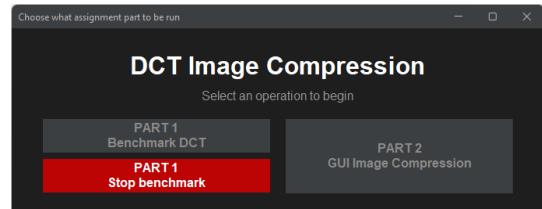
I metodi di conversione tra `BufferedImage` e matrici di `double` e il salvataggio in formato BMP sono implementati nella classe `ImageUtils`.

3.2.1 GUI

Il package `com.unimib.assignment.GUI` offre 3 schermate principali: due schermate iniziali per il lancio dei benchmark e per l'avvio della compressione delle immagini, una schermata dedicata alla compressione e una schermata per l'inserimento dei parametri di compressione. La Figura 3.2.1 mostra la schermata iniziale per la scelta tra benchmark e compressione, la Figura 3.2.1 la schermata iniziale una volta scelta l'opzione benchmark, la Figura 3.2 la schermata di compressione delle immagini e la Figura 3.3 la schermata d'inserimento dei parametri di compressione. Per l'avvio dei benchmark e della compressione delle immagini, la UI si appoggia alla classe `PartsLauncher` del package `backend.launcher`.



Schermata iniziale per la scelta tra benchmark e compressione delle immagini



Schermata iniziale una volta scelta l'opzione di benchmark



Figura 3.2: Schermata di compressione delle immagini

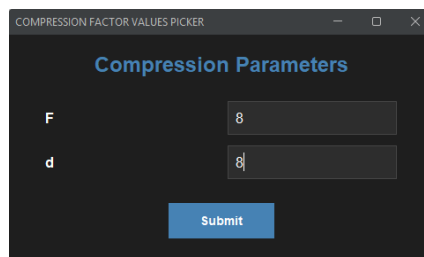


Figura 3.3: Schermata d'inserimento dei parametri di compressione

Bibliografia

- [1] P. Wendykier, *JTransforms*, ver. 3.1, Open-source multithreaded FFT library written in pure Java, 2015. indirizzo: <https://github.com/wendykierp/JTransforms>
- [2] P. Abeles, *Efficient Java Matrix Library (EJML)*, ver. 0.44.0, Fast and easy-to-use linear algebra library written in Java, 2025. indirizzo: <https://github.com/lessthanoptimal/ejml>
- [3] OpenJDK, *JMH: Java Microbenchmark Harness*, Accessed: 2026-05-25, 2026. indirizzo: <https://github.com/openjdk/jmh>
- [4] IBM, *How the JIT Compiler Optimizes Code*, Accessed: 2026-05-25, 2026. indirizzo: <https://www.ibm.com/docs/en/sdk-java-technology/8?topic=compiler-how-jit-optimizes-code>
- [5] P. Wendykier, *DoubleDCT_2D (JTransforms API Documentation)*, https://javadoc.io/doc/com.github.wendykierp/JTransforms/latest/org/jtransforms/dct/DoubleDCT_2D.html, Accessed: 2026-05-27, 2026.
- [6] OpenCSV Project Contributors, *OpenCSV: A Simple CSV Parser for Java*, <https://opencsv.sourceforge.net/>, Version 5.12.0, accessed 2026-05-30, 2025.